Neural Network Application in Traffic Management

Syed Ahmad Shah
Masters of Applied Machine Learning
Stevens Institute of Technology
Hoboken, USA
ashah@stevens.edu

Christina Cuneo
Masters of Applied Machine Learning
Stevens Institute of Technology
Hoboken, USA
ccuneo@stevens.edu

***Abstract*—In Urban Transportation, adaptive traffic signal control is an ongoing challenge, in order to effectively respond to constantly changing demands. This paper presents an approach to using reinforcement learning to select traffic signal phases at a simulated four-way intersection using a Deep-Q Network. The simulation models a four direction intersection, each with three lanes for left-turning, straight, and right-turning, and four signal phases. The agent receives a ten-dimensional state vector encoding per-approach queue lengths, cumulative wait times, the active phase, and time elapsed in that phase. The reward function rewards vehicle throughput while penalizing total wait time and queue length. The model was trained over 1,000 episodes with epsilon-greedy exploration. In early training the average episode reward was -3000 and later -900, showing measurable improvement. Upon analysis a key limitation of the trained model was revealed: : aggregating per-lane counts into per-approach totals causes the agent to misattribute left-turn queue pressure as general congestion, leading to suboptimal phase selection. Future work on this project will address this by changing how lane-level state is represented and by architectural refinement.**

I. Background

Traffic management encompasses monitoring, coordinating, and controlling the movement of vehicles (including private, public transit, and utility vehicles), pedestrians, cyclists, and other road users so that transportation networks operate safely, efficiently, and reliably. In urban environments, this is especially challenging because traffic patterns are continuously subject to change due to congestion, peak hour demand fluctuations, turning movements, pedestrian crossings, transit and associated wait times, roadway incidents, and the presence of high-occupancy and emergency vehicles.

One of the most important tools in traffic management is traffic signal control. The timing and coordination of traffic lights directly govern vehicle flow, intersection delay, pedestrian safety, queue lengths, and the overall performance of the roadway network. Signal control elements such as protected turn signals, pedestrian crossing intervals, and emergency vehicle preemption directly play a critical role in reducing conflict points and mitigating bottleneck formation. Suboptimal signal timing results in increased vehicle delay, network congestion, excess fuel consumption, and widespread gridlock conditions.

Traditional traffic control methods have historically relied on fixed timing schedules or rule-based actuation strategies. While these approaches perform adequately under stable and predictable demand conditions, they are ill-suited to environments characterized by rapid, unanticipated demand shifts or substantial inter-intersection variability. For example, a corridor of intersections may exhibit stable behavior during peak commuting periods, but becomes significantly less so during special events, construction-related detours, or unanticipated shifts in pedestrian volumes. These limitations motivate the development of more intelligent and adaptive traffic control methodologies.

Modern machine learning techniques, and neural networks in particular, have gained substantial traction in this domain because they can learn from large amounts of traffic data and identify complex, non-linear patterns that can resist characterization by conventional analytical methods. These models can leverage the learned representations to make predictions, support decision-making, and facilitate real-time adjustments to traffic signal operations. Applied to traffic management, neural networks are capable of continuously analyzing changing conditions and creating more adaptive, data-driven signal control strategies.

This capacity is especially valuable during disruptions to nominal traffic behavior, such as incidents, active construction zones, special events, or sudden surges in demand. In these scenarios, adaptive control models may respond more effectively than fixed signal timing systems by dynamically adapting to real-time conditions. As a result, neural-network-based approaches have considerable promise for improving network efficiency, reducing congestion, and enabling smarter traffic signal control in complex urban environments.

II. System Architecture

The project has been organized as an adaptive traffic signal controller that utilizes a reinforcement learning pipeline. Within the repository:

- `code/` Contains the main implementation
- `doc/` Intended for reports and project progress
- `references/` Supporting material
- `assets/` For miscellaneous items, images, plots, etc.

Within the `code/` directory, there exist three directories that are separated by task/purpose:

- `State/` Defines the traffic environment, legal moves, state representation
- `Neural_Networks/` Contains various implementations of Neural Networks [Classical NN, DQN, etc]
- Contains trained models separated by [model, epoch iterations]

- `Simulation/` Evaluations of the trained model in different situations and environments
- Visual representation of a 4-way intersection, to provide a better view of the environment and state

We first define the traffic control problem as a simplified environment with four approaches and four signal phases (more may be added later). Think of an intersection with four lanes labeled [a, b, c, d], where the signal phases then become [AC_Forward, BD_Forward, AC_Left, BD_Left]. The input to the model is shown below:

	approach	left_queue	straight_queue	right_queue	total_queue	wait_time_s	current_phase	time_in_phase_s
0	A	1	2	1	4	6	BD_forward	24
1	B	9	1	1	11	28	BD_forward	24
2	C	1	2	1	4	7	BD_forward	24
3	D	8	2	1	11	26	BD_forward	24

Fig. 1. Raw state inputs collected from each approach.

Where we then normalize the input to a 10-value state vector:

	A_queue	B_queue	C_queue	D_queue	A_wait	B_wait	C_wait	D_wait	current_phase	time_in_phase
0	0.2	0.55	0.2	0.55	0.12	0.56	0.14	0.52	0.25	0.48

Fig. 2. Normalized 10-dimensional state vector passed to the DQN.

Finally, the trained model then outputs the selected phase to perform:

	phase_id	phase	q_value	selected
0	0	AC_forward	-1377.950317	False
1	1	BD_forward	-1347.075684	True
2	2	AC_left	-1416.123657	False
3	3	BD_left	-1398.351562	False

Fig. 3. Model output: selected signal phase for the current step.

The architecture keeps a focus on keeping the environment, learning model, and simulation implementations separate and consistent, to allow us to easily iterate the focus of our solution if required.

III. Design and Algorithm

For our first iteration we have decided to implement a Deep Q-Learning model to select the best traffic signal phase at each decision point. The environment first starts with random queue counts and zero wait times, simulates the effect a phase would have on the status, and returns the appropriate next state and reward. The four phases represent the 4 legal actions the model can choose from, this was implemented to reduce the search space for the Deep Q-Network to detect and learn from, and to prevent catastrophic illegal actions. Currently, for every forward phase only two vehicles are allowed to pass, and for left-turn phases only one vehicle is allowed to pass. This will be improved upon by researching the average time it takes a vehicle to cross an intersection, utilizing time as our restriction rather than purely the number of vehicles.

The DQN itself is relatively lightweight, as a fully connected feedforward Neural Network, with two hidden layers and ReLU activations. The optimization utilizes a learning rate of 0.001, with replay batches of 64, and a decaying epsilon from 1.00 to 0.05. During training, the model would likely choose the best known action from its memory as the next move, however with our probability "epsilon" we force it to pick a random action so it can explore other possibilities.

The reward function is designed to encourage flow, and penalize congestion. We combine the negative total wait time across all lanes, the negative queue size, and a positive reward for cars that pass through the intersection:

$$R = -\sum_{i=1}^{4} W_i - 0.5 \sum_{i=1}^{4} Q_i + 2P \tag{1}$$

where

$$W_i = \text{wait time for an approach } i \tag{2}$$

$$Q_i = \text{queue for an approach } i \tag{3}$$

$$P = \text{number of cars that passed during that step} \tag{4}$$

This motivates the model to minimize the delay and queue, rather than just maximizing the number of cars that pass, or the time of a light, thus improving throughput.

IV. Testing and Simulation Results

The model was trained over 1000 episodes against the simulated environment. As shown in Fig. 4, episode reward improves substantially from approximately -3000 in early training to under -900 by episode 1000, demonstrating that the agent learns to reduce congestion and wait times over time. Per-episode variance remains high due to the stochastic arrival of vehicles, but the upward trend in the smoothed average confirms consistent policy improvement.

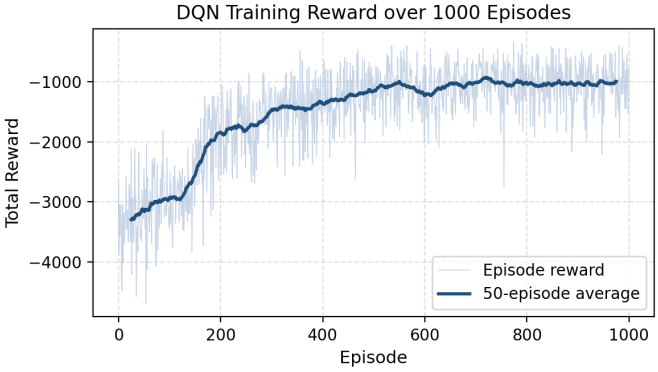

Fig. 4. DQN training reward over 1000 episodes. The 50-episode moving average (dark line) shows steady improvement from approximately -3000 in early episodes to under -900 by episode 1000, despite high per-episode variance.

However, this approach has a critical limitation: the model only sees queue totals by approach and does not differentiate

between lanes. This results in left-lane queue pressure being misinterpreted as total lane pressure, which may cause the model to prefer “Forward” phases to maximize cars passed rather than the more appropriate left-turn phase.

Constraint: 2 cars allowed forward at a time, 1 car allowed to turn left at a time

Visually Represented:

10 left lane ; 1 forward lane

Model interprets as 12 in the B approach, and proceeds to minimize the number of cars in the approach B, by choosing the action that would result in more cars exiting the approach (learned from training), thus it would choose BD_Forward, where the total queue count would become 11.

This was observed in our testing:

Input:

	approach	left_queue	straight_queue	right_queue	total_queue	wait_time_s	current_phase	time_in_phase_s
0	A	1	2	1	4	6	BD_forward	24
1	B	9	1	1	11	28	BD_forward	24
2	C	1	2	1	4	7	BD_forward	24
3	D	8	2	1	11	26	BD_forward	24

Fig. 5. Model input showing a heavily left-turn-weighted queue in the BD approach.

Output:

	phase_id	phase	q_value	selected
0	0	AC_forward	-1377.950317	False
1	1	BD_forward	-1347.075684	True
2	2	AC_left	-1416.123657	False
3	3	BD_left	-1398.351562	False

Fig. 6. Model incorrectly selects BD_Forward despite left-turn queue pressure.

Here, the BD approach in their left_queue has a total of 17 cars, with only 5 cars in the other lanes. However, we provide the total_queue, thus the model determines the best action is BD_Forward instead of BD_Left.

To observe the model’s state and action at different time intervals, we produced a visual graphical implementation to better understand the model’s decisions and mistakes.

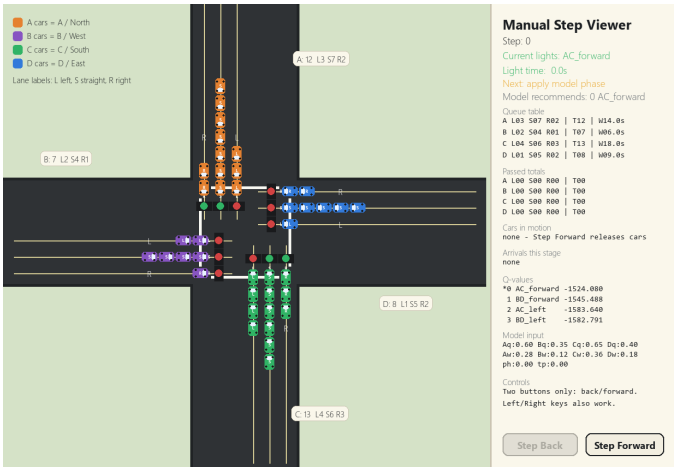

Fig. 7. Pygame visualizer showing intersection state, per-lane queue counts, and active signal phases.

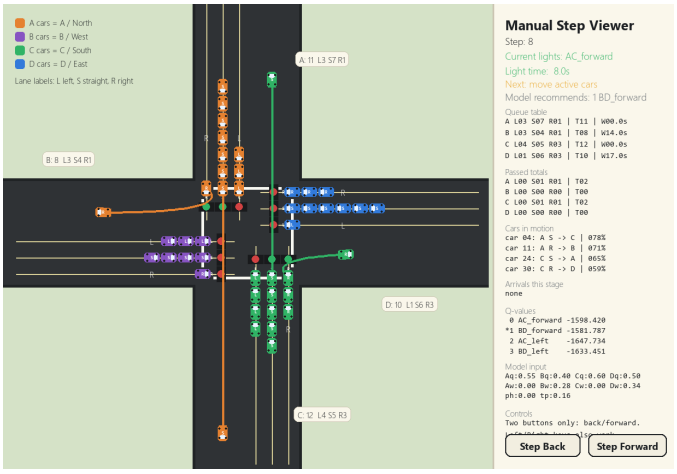

Fig. 8. Visualizer side panel displaying Q-values, model input vector, and cars in motion.

V. Conclusion

This work demonstrates that a lightweight DQN can learn an adaptive signal control policy for a simulated four-way intersection, reducing average episode reward from approximately -3000 to under -900 over 1000 training episodes. The trained model consistently prioritizes high-pressure approaches and avoids prolonged single-phase repetition. A visual step-through tool was developed to inspect model decisions at each stage, which directly enabled identification of the key limitation described below.

The most impactful avenue for improvement in the neural network lies in expanding the state representation to incorporate lane-level vehicle counts and waits as distinct input rather than aggregating them into a single composite value. This granularity enables the model to better capture directional flow and imbalances across individual lanes. This allows for more precise and context-aware signal time decisions, particularly at high-volume or more complex intersections (additional lanes, more than basic four-way intersections, etc). Future work will

focus on refining the architectural complexity of the neural network, through the introduction of regularization techniques and dropout layers. These will serve to prevent the model from overfitting due to training noise.

REFERENCES

[1] TrafficSimulationOptimizationConsideringDrivingStyles-Sciencedirect, www.sciencedirect.com/science/article/pii/S2772424725000216. Accessed28Apr.2026.

[2] President,KLDAssociates,Inc.300Broadway,HuntingtonStation, NY1174618,www.fhwa.dot.gov/publications/research/operations/tft/ chap10.pdf.Accessed28Apr.2026.

[3] EnergyConsumptionandFaultAnalysisofDataCenterPowerSystem| IEEEConferencePublication|IEEEXplore,ieeexplore.ieee.org/document/ 10245678/.Accessed28Apr.2026.

[4] (PDF)ResearchonOptimizationAlgorithmforUrbanTrafficFlowBasedonComputerSimulation, www.researchgate.net/publication/375849235_Research_on_ Optimization_Algorithm_for_Urban_Traffic_Flow_Based_on_ Computer_Simulation.Accessed28Apr.2026.

[5] Deng,Xuefeng,etal.âĂœTrafficFlowSimulationofModifiedCellularAutomataModelBasedonProducer-ConsumerAlgorithm. âĂİPeerJ.ComputerScience,U.S.NationalLibraryofMedicine,20Sept. 2022,pmc.ncbi.nlm.nih.gov/articles/PMC9575854/.